

- 一、Electron为什么存在
- 二、Main Process（主进程）
- 三、Renderer Process（渲染进程）
- 四、IPC通信
- 五、Preload（预加载脚本）
- 六、BrowserWindow
- 七、WebContents
- 八、Node Integration
- 九、Context Isolation
- 十、你们项目最可能遇到的架构

对于你明天的实习来说，不需要成为 Electron 专家，但有几个核心概念必须知道。否则导师讲项目架构时，你会听得比较吃力。

Electron 本质上是：

用 Web 技术（HTML、CSS、JavaScript、Vue）开发桌面应用。

你可以把它理解成：

```
Chromium浏览器
+
Node.js运行时
+
桌面应用能力
=
Electron
```

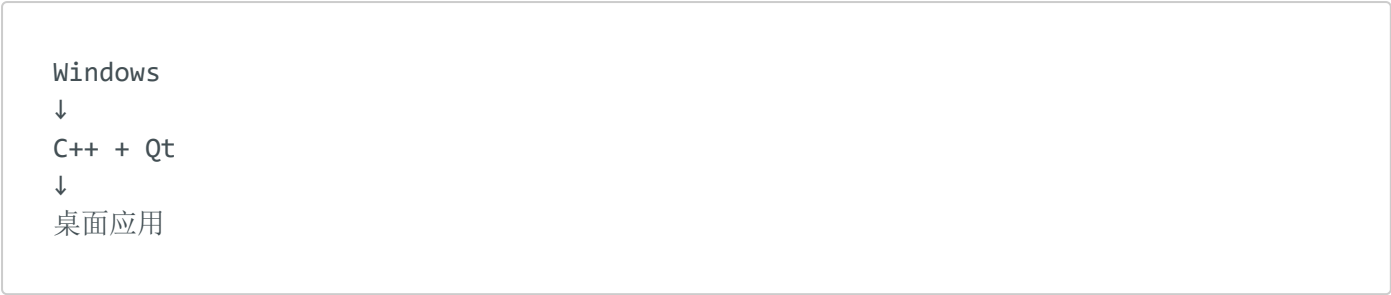
常见的软件：

- VS Code
- Discord
- Notion
- Obsidian
- Postman

都使用过 Electron。

一、Electron为什么存在

传统桌面软件：



你们团队以前就是这样。

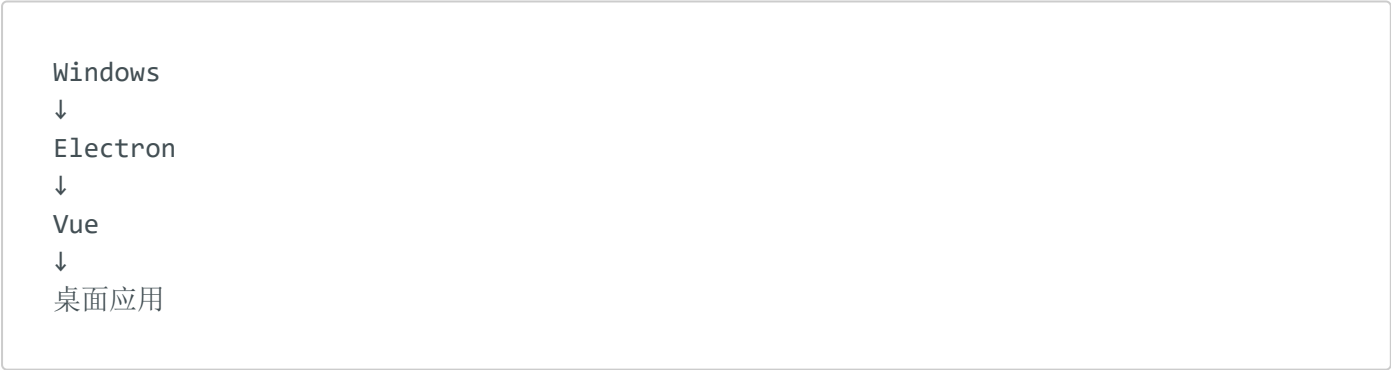
优点：

- 性能好
- 系统能力强

缺点：

- 前端界面开发效率低
- 招人难
- 前端生态无法直接使用

Electron模式：



优点：

- 前端开发快
- Vue生态直接复用
- 招前端工程师容易

所以你们项目才会：



技术转型。

二、Main Process（主进程）

Electron最重要的概念。

可以理解为：

应用的大脑

负责：

- 创建窗口
- 管理窗口
- 操作文件系统
- 操作系统功能
- 管理生命周期

例如：

```
const { BrowserWindow } = require('electron')  
  
const win = new BrowserWindow({  
  width: 1200,  
  height: 800  
})
```

这里就是主进程在创建窗口。

在项目里通常会看到：

```
main.js  
background.js  
electron/main/index.js
```

之类的文件。

这些大概率是主进程入口。

三、Renderer Process（渲染进程）

你以后主要写的代码。

可以理解为：

浏览器页面

例如：

Vue 页面
组件
路由
状态管理
业务逻辑

都运行在渲染进程。

结构类似：

src
├ views
├ components
├ router
├ store
└ api

基本和普通Vue项目一样。

关系图：



```
└─ Renderer Process
    │
    ├── Vue
    ├── HTML
    ├── CSS
    └─ JavaScript
```

四、IPC通信

这是面试和工作中最常见的概念。

IPC:

```
Inter-Process Communication
进程间通信
```

为什么需要？

因为：

```
Main Process
≠
Renderer Process
```

是两个不同进程。

例如：

Vue页面想读取本地文件。

直接这样：

```
fs.readFile(...)
```

很多情况下不允许。

需要：

Vue页面
↓
发送消息
↓
Main Process
↓
读取文件
↓
返回结果
↓
Vue页面

这就是IPC。

发送消息：

```
ipcRenderer.send(...)
```

主进程接收：

```
ipcMain.on(...)
```

现代写法：

```
ipcRenderer.invoke(...)
```

```
ipcMain.handle(...)
```

你以后看到：

```
window.electron.xxx()
```

背后大概率也是IPC。

五、Preload（预加载脚本）

这是新人最容易懵的概念。

Electron早期：

```
require('fs')
```

可以直接在页面执行。

后来发现：

太危险

因为页面被攻击后：

直接拥有系统权限

所以现在通常是：

```
Renderer  
↓  
Preload  
↓  
Main
```

三层结构。

Preload作用：

安全中间层

例如：

```
contextBridge.exposeInMainWorld(  
  'electronAPI',  
  {  
    openFile: () => ipcRenderer.invoke('open-file')  
  }  
)
```

然后Vue里：

```
window.electronAPI.openFile()
```

就能调用。

你明天如果看到：

```
window.xxx
```

大概率就是Preload暴露的接口。

六、BrowserWindow

Electron窗口对象。

创建窗口：

```
new BrowserWindow({  
  width:1200,  
  height:800  
})
```

实际对应：

一个桌面窗口

例如：

- 主界面
- 登录窗口
- 设置窗口

都属于 BrowserWindow。

七、WebContents

窗口里的网页内容。

例如：

```
win.webContents.send(...)
```

作用：

主进程给页面发消息

很多项目会这样：

```
设备状态变化
↓
主进程收到
↓
webContents.send
↓
Vue界面刷新
```

工业软件经常这样用。

八、Node Integration

决定页面是否能直接使用Node能力。

例如：

```
require('fs')
```

开启：

```
nodeIntegration:true
```

页面能直接访问Node。

关闭：

```
nodeIntegration:false
```

需要通过Preload。

现代项目基本推荐关闭。

所以你们新项目大概率是：

```
nodeIntegration:false
```

九、Context Isolation

也是安全相关。

通常会看到：

```
contextIsolation:true
```

作用：

网页环境
和
Electron环境
隔离

现代项目一般：

```
nodeIntegration:false  
contextIsolation:true
```

这是比较标准的配置。

十、你们项目最可能遇到的架构

结合导师介绍：

```
Qt旧客户端  
↓  
功能迁移  
↓  
Electron  
↓  
Vue2
```

我猜项目结构大概率类似：

```
electron  
├─ main  
│   ├── main.js  
│   └─ ipc  
├─ preload  
│   └─ preload.js  
└─ src  
    ├── views  
    ├── components  
    ├── store  
    ├── router  
    └─ api
```

其中：

main

负责系统能力

preload

负责桥接

src

负责业务页面

你作为实习生，前两周 80% 的时间大概率都在：

src/views

src/components

里面写 Vue2 页面。

但随着项目深入，你迟早会接触：

IPC

Preload

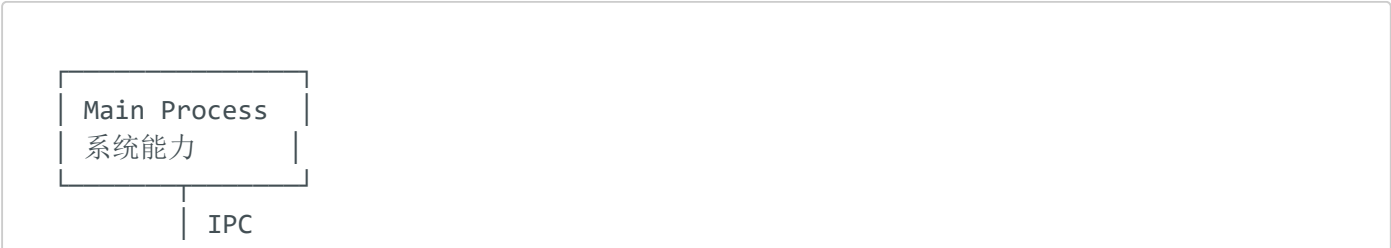
Main Process

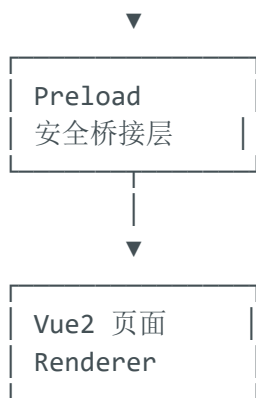
因为工业物联网客户端经常需要：

- 本地文件
- 串口
- 网络设备
- 日志
- 自动更新
- 本地配置

这些都离不开 Electron 主进程。

你今晚只需要牢牢记住这一张图：





明天导师如果讲 Electron 架构，你能理解这三个层级以及它们之间的通信关系，基本就不会跟丢。